

# Functional Requirements Document (FRD)

## Bharat MCP — Restaurant MCP Aggregator, Bangalore Pilot

**Document version:** 1.0 **Date:** September 15, 2026 **Prepared for:** Reward360 Founder’s Office — internal, discreet **Scope of this release:** 500-restaurant Bangalore pilot **Proposed stack:** React + TypeScript (frontend), FastAPI/Python (backend + MCP server), PostgreSQL, Redis

### 1. Introduction

#### 1.1 Purpose

This document defines the functional and non-functional requirements for building Bharat MCP — a system that exposes structured restaurant data (menus, availability, orders, bookings) to AI agents (Claude, ChatGPT, Gemini) via the Model Context Protocol, and provides the internal tooling needed to onboard and manage 500 restaurants in Bangalore for a pilot.

#### 1.2 Scope

In scope: restaurant/menu data management, an internal admin tool for onboarding, an MCP server exposing tools to AI agents, order creation and relay to restaurants without a POS, UPI payment collection, and monitoring.

Out of scope for this release: POS integrations, table-booking beyond a basic implementation, multi-city expansion, restaurant-side native app, loyalty/rewards tie-in to R360’s own platform (may be a future integration point, not pilot scope).

#### 1.3 Definitions

- MCP** — Model Context Protocol; the open standard by which AI agents call external tools/data sources.
- Tenant** — a single restaurant’s record within the shared multi-tenant system.
- Agent** — an AI assistant (Claude, ChatGPT, Gemini) acting on an end user’s behalf.
- Relay** — the mechanism (WhatsApp/SMS) by which a confirmed order reaches a restaurant with no POS integration.

#### 1.4 Stakeholders

- Infra/engineering lead — owns build and delivery
- 2 interns — restaurant onboarding, data entry, first-line QA
- Founder’s office (Amit) — sponsor, external partner conversations (AI platforms, POS vendors)
- Restaurant owners — end operators who must accept and fulfill orders
- AI platforms (Anthropic, OpenAI, Google) — consumers of the MCP interface

### 2. Overall Description

#### 2.1 Product Perspective

Bharat MCP is a new, standalone system. It is not a modification of an existing R360 product, though it may eventually integrate with R360’s reward/loyalty stack (out of scope for pilot).

#### 2.2 User Classes

| User class              | Description                                                         | Access                                                   |
|-------------------------|---------------------------------------------------------------------|----------------------------------------------------------|
| Admin (interns/lead)    | Adds/edits restaurants, menus, monitors orders                      | Full admin panel access                                  |
| Restaurant owner        | Receives orders via WhatsApp/SMS, accepts/rejects/fulfills          | No login required for pilot — relay-only                 |
| AI agent (via platform) | Calls MCP tools on behalf of an end user                            | API-key/OAuth-scoped MCP access, read + order-write only |
| End consumer            | Interacts only through the AI agent, never directly with Bharat MCP | No direct access                                         |

#### 2.3 Assumptions & Constraints

- Most pilot restaurants have no POS system; order relay is manual-confirmation via WhatsApp/SMS, not real-time system-to-system.
- Single Bangalore-only pilot; no multi-region data residency concerns for this phase.
- Team is 1 infra lead + 2 interns — the admin tool must let non-engineers do 100% of restaurant onboarding without code changes.
- Menu photographs/PDFs must be manually structured into the schema; no OCR automation in scope for pilot (may be a fast-follow).

### 3. Functional Requirements

3.1 Restaurant & Menu Management (Admin Panel — React/TS frontend)

| ID   | Requirement                                                                                                                         | Priority | Acceptance Criteria                                                                                         |
|------|-------------------------------------------------------------------------------------------------------------------------------------|----------|-------------------------------------------------------------------------------------------------------------|
| FR-1 | Admin can create a new restaurant record (name, address, geo-coordinates, phone, WhatsApp number, cuisine tags, hours)              | Must     | Record saved to Postgres; required fields enforced; duplicate phone/name+address flagged                    |
| FR-2 | Admin can add/edit menu categories and items (name, description, price, veg/non-veg, image, availability) under a restaurant        | Must     | Items correctly nested under categories; price must be numeric > 0; image upload succeeds to object storage |
| FR-3 | Admin can mark an item or the whole restaurant temporarily unavailable (sold out, closed)                                           | Must     | Reflected in <code>get_menu</code> / <code>check_availability</code> MCP responses within 1 minute          |
| FR-4 | Admin can record owner consent (opt-in) before a restaurant goes live                                                               | Must     | Restaurant cannot transition to <code>active</code> status without a consent flag set                       |
| FR-5 | System runs an automated smoke test (test-mode <code>get_menu</code> + <code>place_order</code> call) before a restaurant goes live | Must     | Restaurant blocked from <code>active</code> status if smoke test fails                                      |
| FR-6 | Admin can view a restaurant's live status (pending/active/paused) and order history                                                 | Should   | Status and last 50 orders visible on restaurant detail page                                                 |
| FR-7 | Admin can bulk-import restaurants/menus via CSV for faster onboarding                                                               | Could    | CSV template provided; validation errors reported per row, not just per file                                |

3.2 MCP Tool Surface (exposed to AI agents)

| ID    | Tool                                                                           | Requirement                                                     | Priority | Acceptance Criteria                                                                           |
|-------|--------------------------------------------------------------------------------|-----------------------------------------------------------------|----------|-----------------------------------------------------------------------------------------------|
| FR-8  | <code>search_restaurants(location, cuisine, query)</code>                      | Returns matching active restaurants in Bangalore pilot coverage | Must     | Results limited to <code>active</code> status; response time < 1.5s p95                       |
| FR-9  | <code>get_menu(restaurant_id)</code>                                           | Returns structured categories/items/prices/availability         | Must     | Reflects latest admin edits within 1 minute; unavailable items flagged, not omitted           |
| FR-10 | <code>check_availability(restaurant_id, item_ids, time)</code>                 | Real-time stock/hours check                                     | Must     | Correctly rejects orders outside operating hours                                              |
| FR-11 | <code>place_order(restaurant_id, items, customer_contact, payment_mode)</code> | Creates an order, triggers relay                                | Must     | Idempotency key required; duplicate calls with same key return same order, not a second order |
| FR-12 | <code>get_order_status(order_id)</code>                                        | Returns current order state                                     | Must     | States: placed / accepted / rejected / fulfilled / cancelled, each with timestamp             |
| FR-13 | <code>book_table(restaurant_id, party_size, time)</code>                       | Table reservation (if in pilot scope)                           | Should   | Only enabled for restaurants flagged as accepting bookings                                    |

3.3 Order Relay & Fulfillment

| ID    | Requirement                                                                                                                    | Priority | Acceptance Criteria                                                                         |
|-------|--------------------------------------------------------------------------------------------------------------------------------|----------|---------------------------------------------------------------------------------------------|
| FR-14 | On order placement, system sends a WhatsApp (or SMS fallback) message to the restaurant's registered number with order details | Must     | Message delivered within 30 seconds of order creation                                       |
| FR-15 | Restaurant owner can accept/reject an order via a simple reply (structured keyword or link)                                    | Must     | Reply correctly updates order status; ambiguous replies flagged for manual intern follow-up |
| FR-16 | System notifies the AI agent (via <code>get_order_status</code> or webhook) when status changes                                | Must     | Status change reflected within 1 minute of restaurant reply                                 |
| FR-17 | Orders unconfirmed after a                                                                                                     | Should   | Timeout configurable per restaurant;                                                        |

|  |                                                                                  |  |                                       |
|--|----------------------------------------------------------------------------------|--|---------------------------------------|
|  | configurable timeout (e.g., 10 minutes) are auto-flagged for intern intervention |  | flagged orders visible in admin panel |
|--|----------------------------------------------------------------------------------|--|---------------------------------------|

### 3.4 Payments

| ID    | Requirement                                                                                    | Priority | Acceptance Criteria                                                                                |
|-------|------------------------------------------------------------------------------------------------|----------|----------------------------------------------------------------------------------------------------|
| FR-18 | System generates a UPI payment intent/deep link via PSP (Razorpay/Cashfree) for prepaid orders | Must     | Payment link valid, expires after a configurable window, order marked paid on webhook confirmation |
| FR-19 | System supports cash-on-delivery as a fallback payment mode                                    | Must     | COD orders flow through the same order lifecycle without a payment step blocking fulfillment       |
| FR-20 | Payment webhook failures are retried and logged, not silently dropped                          | Must     | Failed webhook triggers retry (min 3 attempts) and an alert if all fail                            |

### 3.5 AI Platform Integration

| ID    | Requirement                                                                                    | Priority | Acceptance Criteria                                                                         |
|-------|------------------------------------------------------------------------------------------------|----------|---------------------------------------------------------------------------------------------|
| FR-21 | MCP server is registered and authenticated against each target AI platform's connector process | Must     | Successful end-to-end test call from each platform (Claude, ChatGPT, Gemini)                |
| FR-22 | Tool-level errors return agent-readable messages, not raw exceptions                           | Must     | Verified via test transcripts covering unavailable item, closed restaurant, payment failure |
| FR-23 | API keys/tokens per platform are rotatable without downtime                                    | Should   | Key rotation does not interrupt in-flight requests                                          |

### 3.6 Monitoring & Admin Visibility

| ID    | Requirement                                                                                                      | Priority | Acceptance Criteria                                                |
|-------|------------------------------------------------------------------------------------------------------------------|----------|--------------------------------------------------------------------|
| FR-24 | System logs errors to a central error tracker (e.g., Sentry)                                                     | Must     | Errors visible with restaurant/order context, not just stack trace |
| FR-25 | Uptime monitoring alerts within 5 minutes of MCP endpoint downtime                                               | Must     | Alert delivered to lead via configured channel (email/Slack/SMS)   |
| FR-26 | Admin panel shows a daily summary: restaurants onboarded, orders placed, orders fulfilled, orders failed/flagged | Should   | Dashboard reflects previous 24h rolling window                     |

## 4. External Interface Requirements

### 4.1 REST API (Admin Panel ↔ FastAPI backend)

Standard authenticated REST endpoints under `/api/v1/`: restaurants (CRUD), menu-items (CRUD), orders (read, status update), onboarding (consent + smoke-test trigger). All endpoints require an admin session token; request/response bodies validated via Pydantic schemas shared with the MCP tool layer.

### 4.2 MCP Interface (FastAPI ↔ AI platforms)

Exposed via Streamable HTTP transport per the MCP specification, mounted as a sub-application within the same FastAPI service. Tool schemas (input/output) are defined once in Pydantic and reused for both the MCP layer and internal validation — no duplicate schema maintenance.

### 4.3 Third-Party Interfaces

- **WhatsApp Business API** — outbound order notification, inbound accept/reject parsing
- **Razorpay/Cashfree** — UPI intent generation, payment webhook receipt
- **Object storage (S3-compatible)** — menu image upload/retrieval from the React admin panel

## 5. Non-Functional Requirements

| Category    | Requirement                                                                                                                                                               |
|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Performance | MCP tool responses p95 < 1.5s for read operations ( <code>search_restaurants</code> , <code>get_menu</code> ); < 3s for <code>place_order</code> including relay dispatch |

|                |                                                                                                                                                                                      |
|----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Availability   | Target 99% uptime during pilot business hours (11am–11pm IST); single shared server is an accepted pilot-scale tradeoff, not a production SLA                                        |
| Scalability    | Architecture must support scaling from 500 to 2,000+ restaurants without a schema or protocol redesign — horizontal scaling of compute only, not a data-model rework                 |
| Security       | All traffic over HTTPS; admin panel behind authenticated sessions with role-based access; API keys scoped per AI platform; no card/UPI credentials stored directly (PSP-hosted only) |
| Data integrity | <code>place_order</code> must be idempotent; no duplicate orders from retried agent calls                                                                                            |
| Auditability   | Owner consent, order state transitions, and admin edits to menu/pricing are timestamped and attributable to a user/system actor                                                      |
| Usability      | Admin panel usable by interns with no engineering background after a single walkthrough — no direct database or code access required for any onboarding task                         |

## 6. Data Requirements

Core entities (see infra plan for full field-level schema): `restaurants`, `menu_categories`, `menu_items`, `availability_overrides`, `orders`, `bookings`. Relationships: one restaurant → many categories → many items; one restaurant → many orders; one order → many order line items (referencing `menu_items` at time of order, price-snapshotted to avoid retroactive price changes affecting historical orders).

## 7. System Architecture Summary

- **Frontend (admin panel):** React + TypeScript, calling the FastAPI REST layer over authenticated HTTPS
- **Backend:** FastAPI (Python), hosting both the REST API and the MCP server (mounted as an ASGI sub-app), using async SQLAlchemy + asyncpg for Postgres and redis-py/aioredis for Redis
- **Database:** PostgreSQL (managed — RDS/Supabase/Neon)
- **Cache/session:** Redis (managed — Upstash)
- **Object storage:** S3-compatible (Cloudflare R2)
- **Relay:** WhatsApp Business API, SMS fallback
- **Payments:** Razorpay or Cashfree
- **Monitoring:** Sentry + uptime monitoring

(Full component rationale and cost estimate covered in the separately delivered infra plan — this FRD focuses on functional requirements and acceptance criteria; refer to that document for hosting sizing and cost breakdown.)

## 8. Acceptance Criteria for Pilot Sign-Off

The pilot is considered functionally complete when: 1. 500 restaurants are onboarded with owner consent recorded and smoke tests passed. 2. At least one successful end-to-end order (search → menu → order → relay → fulfillment → payment) is demonstrated on each of Claude, ChatGPT, and Gemini. 3. Order relay reliably reaches restaurants (measured: >90% of orders receive a restaurant response within the configured timeout). 4. Admin panel requires zero engineering intervention for standard onboarding across a full week of intern use. 5. No unresolved critical (data-loss or duplicate-charge) defects remain open.

## 9. Out of Scope (This Release)

POS system integrations; multi-city rollout; native restaurant-facing app; automated menu OCR; loyalty-points integration with R360’s existing platform; multi-language menu support.

## 10. Open Questions for Engineering Kickoff

- Final choice between Razorpay and Cashfree (commercial terms, integration effort)
- Whether table booking (FR-13) is in or out of pilot scope
- WhatsApp Business API provider (direct Meta onboarding vs. a BSP like Gupshup/Twilio) — affects relay setup time